

199-assignment-bayesian-statistics

February 26, 2026

0.0.1 Adeline Makokha

0.0.2 191199

0.0.3 DSA 8505 Bayesian Statistics

0.0.4 Assignment

Title Comparative Analysis of Classical OLS and Bayesian Multiple Regression in House Price Prediction

####1. Dataset Selection:

Identify an appropriate dataset online (e.g., from Kaggle, UCI Machine Learning Repository, or government open data portals). The dataset must have multiple independent variables (predictors) and at least one continuous dependent variable.

####Introduction

Accurate estimation of residential property prices is essential for informed decision-making in real estate markets, mortgage lending, urban planning, and property investment. Housing prices are influenced by numerous structural and neighborhood characteristics, including property size, quality, amenities, and location. Given this multidimensional nature, statistical modeling provides a systematic approach to understanding how these predictors jointly influence price variation.

####Problem Statement

Property valuation often relies on traditional appraisal methods that may introduce subjectivity and inconsistency. With numerous interrelated predictors, it becomes difficult to quantify how each housing characteristic contributes to overall price.

This study addressed the following question:

How can multiple housing features be used to construct a reliable predictive model for house prices?

How does Bayesian regression compare to classical OLS in terms of parameter estimation, uncertainty quantification, and predictive accuracy?

####Objective of the study

The objective of this study was to model house prices using multiple independent variables and compare two regression frameworks: Classical Ordinary Least Squares (OLS) and Bayesian Multiple Regression. The dependent variable was a continuous outcome (log-transformed sale price), and the analysis focused on predictive performance, parameter estimation, and uncertainty quantification.

Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

```
[2]: #loading data from github
url = "https://raw.githubusercontent.com/adeline-pepela/Module-5/main/
↳Bayesian%20Statistics/house-prices-advanced-regression-techniques/
↳house-price.csv"

df = pd.read_csv(url)
df.head()
```

```
[2]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	\
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	\
0	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	
1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	
2	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	
3	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	
4	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	

	YrSold	SaleType	SaleCondition	SalePrice
0	2008	WD	Normal	208500
1	2007	WD	Normal	181500
2	2008	WD	Normal	223500
3	2006	WD	Abnorml	140000
4	2008	WD	Normal	250000

[5 rows x 81 columns]

2. Data Preprocessing: Load the dataset and perform exploratory data analysis (EDA). Handle missing values, outliers, and categorical variables appropriately. Split the data into training and

testing sets. Classical Multiple

```
[3]: # Dataset shape
print("Shape:", df.shape)
```

Shape: (1460, 81)

This shows the shape of our data of 1460 rows and 81 columns.

```
[4]: # Data types
print("\nData Types:\n", df.dtypes)
```

Data Types:

```
Id          int64
MSSubClass  int64
MSZoning    object
LotFrontage float64
LotArea     int64
...
MoSold      int64
YrSold      int64
SaleType    object
SaleCondition object
SalePrice   int64
Length: 81, dtype: object
```

This shows the various data types.

```
[5]: # Summary statistics
df.describe()
```

```
[5]:
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	\
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	
std	421.610009	42.300571	24.284752	9981.264932	1.382997	
min	1.000000	20.000000	21.000000	1300.000000	1.000000	
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	

	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	\
count	1460.000000	1460.000000	1460.000000	1452.000000	1460.000000	...	
mean	5.575342	1971.267808	1984.865753	103.685262	443.639726	...	
std	1.112799	30.202904	20.645407	181.066207	456.098091	...	
min	1.000000	1872.000000	1950.000000	0.000000	0.000000	...	
25%	5.000000	1954.000000	1967.000000	0.000000	0.000000	...	
50%	5.000000	1973.000000	1994.000000	0.000000	383.500000	...	

75%	6.000000	2000.000000	2004.000000	166.000000	712.250000	...
max	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	...

	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch	\
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	
mean	94.244521	46.660274	21.954110	3.409589	15.060959	
std	125.338794	66.256028	61.119149	29.317331	55.757415	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	25.000000	0.000000	0.000000	0.000000	
75%	168.000000	68.000000	0.000000	0.000000	0.000000	
max	857.000000	547.000000	552.000000	508.000000	480.000000	

	PoolArea	MiscVal	MoSold	YrSold	SalePrice
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	2.758904	43.489041	6.321918	2007.815753	180921.195890
std	40.177307	496.123024	2.703626	1.328095	79442.502883
min	0.000000	0.000000	1.000000	2006.000000	34900.000000
25%	0.000000	0.000000	5.000000	2007.000000	129975.000000
50%	0.000000	0.000000	6.000000	2008.000000	163000.000000
75%	0.000000	0.000000	8.000000	2009.000000	214000.000000
max	738.000000	15500.000000	12.000000	2010.000000	755000.000000

[8 rows x 38 columns]

This shows the summary statistics for numerical variables.

```
[6]: # Missing values count
missing_values = df.isnull().sum()

# Display columns with missing values
missing_values[missing_values > 0]
```

```
[6]: LotFrontage      259
Alley              1369
MasVnrType         872
MasVnrArea          8
BsmtQual           37
BsmtCond           37
BsmtExposure       38
BsmtFinType1       37
BsmtFinType2       38
Electrical          1
FireplaceQu        690
GarageType          81
GarageYrBlt        81
GarageFinish        81
```

```

GarageQual      81
GarageCond      81
PoolQC         1453
Fence          1179
MiscFeature     1406
dtype: int64

```

There is present of missing values in various variables.

Handling Missing Values

```

[52]: # Columns where NA means "None"
none_cols = [
    "Alley", "MasVnrType", "BsmtQual", "BsmtCond", "BsmtExposure",
    "BsmtFinType1", "BsmtFinType2", "FireplaceQu",
    "GarageType", "GarageFinish", "GarageQual", "GarageCond",
    "PoolQC", "Fence", "MiscFeature"
]

for col in none_cols:
    df[col] = df[col].fillna("None")

# Fill LotFrontage by neighborhood median
df["LotFrontage"] = df.groupby("Neighborhood")["LotFrontage"]\
    .transform(lambda x: x.fillna(x.median()))

df["MasVnrArea"] = df["MasVnrArea"].fillna(df["MasVnrArea"].median())

df["GarageYrBlt"] = df["GarageYrBlt"].fillna(0)

# Convert empty strings to NaN
df["Electrical"] = df["Electrical"].replace("", np.nan)

# fill missing with mode
df["Electrical"] = df["Electrical"].fillna(df["Electrical"].mode()[0])

```

```

[15]: # Final check
df["Electrical"].isnull().sum()

```

```

[15]: np.int64(0)

```

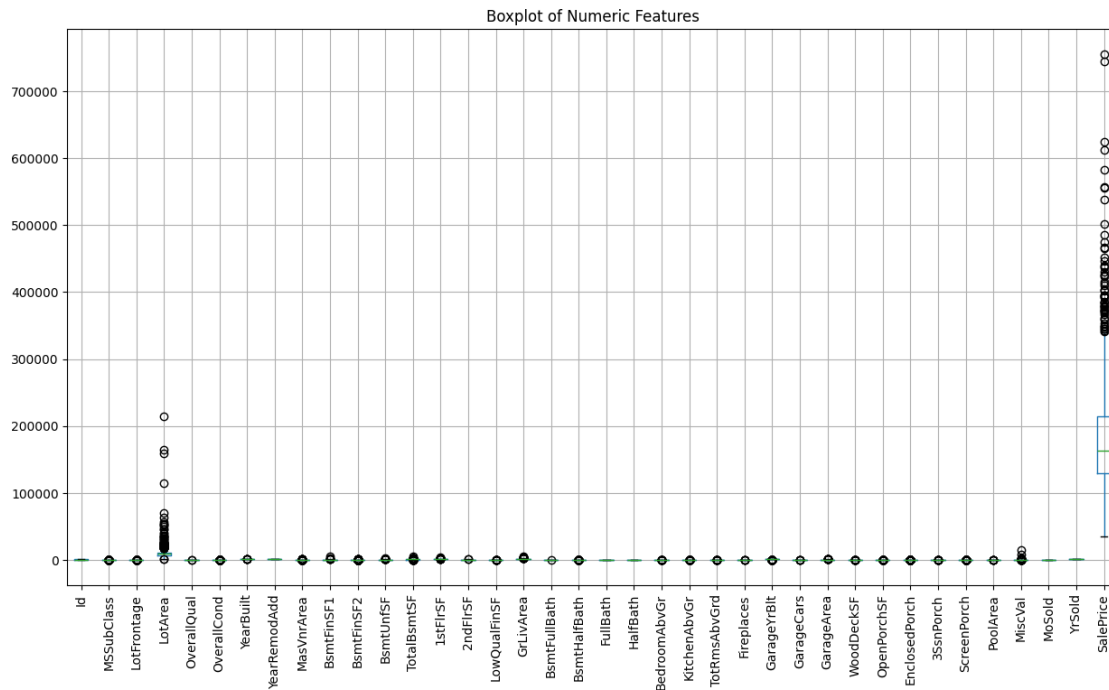
```

[16]: # Select numeric columns
numeric_cols = df.select_dtypes(include=["int64", "float64"]).columns

# Plot boxplots for major numeric variables
plt.figure(figsize=(15,8))
df[numeric_cols].boxplot(rot=90)
plt.title("Boxplot of Numeric Features")

```

```
plt.show()
```



Handling Outliers

```
[53]: # Remove Id column
if "Id" in numeric_cols:
    numeric_cols = numeric_cols.drop("Id")

# Apply IQR filtering
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df = df[(df[col] >= lower) & (df[col] <= upper)]

print("New dataset shape after outlier removal:", df.shape)
```

New dataset shape after outlier removal: (212, 81)

```
[54]: # Check skewness
print("Skewness of SalePrice:", df["SalePrice"].skew())
```

```
# Log-transform target
df["SalePrice"] = np.log1p(df["SalePrice"])
```

Skewness of SalePrice: -0.13388624551958989

Handling Categorical variables

```
[19]: categorical_cols = df.select_dtypes(include=["object"]).columns
      print("Number of categorical variables:", len(categorical_cols))
```

Number of categorical variables: 43

```
[20]: # One-hot encode categorical variables
      df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

      print("Shape after encoding:", df_encoded.shape)
```

Shape after encoding: (516, 184)

```
[21]: # Confirm no categorical variables remain
      df_encoded.select_dtypes(include=["object"]).columns
```

```
[21]: Index([], dtype='object')
```

Separate Features and Target variables

```
[22]: # Define dependent variable
      y = df_encoded["SalePrice"]

      # Define independent variables
      X = df_encoded.drop("SalePrice", axis=1)

      print("Feature matrix shape:", X.shape)
      print("Target vector shape:", y.shape)
```

Feature matrix shape: (516, 183)

Target vector shape: (516,)

Train-Test Split

```
[23]: from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(
          X,
          y,
          test_size=0.20,          # 20% for testing
          random_state=42         # Ensures reproducibility
      )
```

```
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

Training set size: (412, 183)

Testing set size: (104, 183)

Fit the Model on the training data

```
[24]: # Initialize model
model = LinearRegression()

# Fit model
model.fit(X_train, y_train)
```

[24]: LinearRegression()

```
[25]: print("Intercept:", model.intercept_)
```

Intercept: 4.8102757044269095

The intercept represents the predicted log house price when all predictors equal zero.

```
[26]: # Create DataFrame of coefficients
coefficients = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

coefficients.head(10)
```

```
[26]:
```

	Feature	Coefficient
0	Id	0.000004
1	MSSubClass	-0.001891
2	LotFrontage	-0.000430
3	LotArea	0.000006
4	OverallQual	0.022036
5	OverallCond	0.051686
6	YearBuilt	0.004373
7	YearRemodAdd	0.000713
8	MasVnrArea	-0.000090
9	BsmtFinSF1	0.000047

Each coefficient measures the expected change in the log house price for a one-unit increase in the predictor, holding other variables constant.

Positive coefficient → price increases

Negative coefficient → price decreases

Make Predictions on Test Set

```
[27]: # Predict on test data
y_pred = model.predict(X_test)
```

Predictions for unseen data to evaluate model performance.

Evaluate model performance

```
[55]: from sklearn.metrics import r2_score, mean_squared_error

# Calculate R2
r2 = r2_score(y_test, y_pred)

# Calculate RMSE
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("R2 Score:", round(r2, 4))
print("RMSE:", round(rmse, 4))
```

R² Score: 0.8476

RMSE: 0.1147

R² is the Proportion of variance explained by the model

RMSE is the Average prediction error magnitude

Higher R² of **0.8476** and lower RMSE of **0.1147** indicate better model performance.

3. Regression (Ordinary Least Squares - OLS): Fit a multiple linear regression model using OLS. Check model assumptions (linearity, independence, homoscedasticity, normality). Report parameter estimates, R², and residual analysis.

```
[56]: import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
```

Add Constant (Intercept Term)

```
[30]: # Add intercept to training data
X_train_sm = sm.add_constant(X_train)
X_test_sm = sm.add_constant(X_test)
```

Fit OLS Model

```
[32]: # Convert boolean columns to integers for statsmodels
for col in X_train_sm.select_dtypes(include='bool').columns:
    X_train_sm[col] = X_train_sm[col].astype(int)

# Fit OLS model
ols_model = sm.OLS(y_train, X_train_sm).fit()
```

```
# Display summary
print(ols_model.summary())
```

```

OLS Regression Results
=====
Dep. Variable:          SalePrice    R-squared:                0.960
Model:                  OLS          Adj. R-squared:           0.934
Method:                 Least Squares  F-statistic:              37.07
Date:                  Thu, 26 Feb 2026  Prob (F-statistic):       1.72e-118
Time:                  10:33:53       Log-Likelihood:           584.93
No. Observations:      412           AIC:                     -845.9
Df Residuals:          250           BIC:                     -194.5
Df Model:               161
Covariance Type:       nonrobust
=====
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
-----
Id                4.467e-06    1.15e-05      0.389    0.698    -1.81e-05
2.71e-05
MSSubClass        -0.0019      0.001      -1.649    0.100     -0.004
0.000
LotFrontage       -0.0004      0.001      -0.819    0.414     -0.001
0.001
LotArea           5.828e-06    3.36e-06      1.733    0.084    -7.95e-07
1.25e-05
OverallQual        0.0220      0.009      2.514    0.013      0.005
0.039
OverallCond        0.0517      0.011      4.731    0.000      0.030
0.073
YearBuilt          0.0044      0.002      2.748    0.006      0.001
0.008
YearRemodAdd       0.0007      0.001      1.188    0.236     -0.000
0.002
MasVnrArea        -9.017e-05    7.99e-05     -1.129    0.260     -0.000
6.71e-05
BsmtFinSF1         4.716e-05    2.73e-05      1.728    0.085     -6.6e-06
0.000
BsmtFinSF2         1.158e-11    1.82e-11      0.636    0.525    -2.43e-11
4.74e-11
BsmtUnfSF          -4.45e-05    2.62e-05     -1.700    0.090     -9.6e-05
7.04e-06
TotalBsmtSF        2.663e-06    4.82e-05      0.055    0.956    -9.23e-05
9.76e-05

```

1stFlrSF	0.0002	5.48e-05	2.930	0.004	5.26e-05
0.000					
2ndFlrSF	8.317e-05	4.59e-05	1.812	0.071	-7.22e-06
0.000					
LowQualFinSF	4.094e-12	6.53e-12	0.627	0.531	-8.76e-12
1.69e-11					
GrLivArea	0.0002	3.27e-05	7.452	0.000	0.000
0.000					
BsmtFullBath	0.0038	0.016	0.245	0.807	-0.027
0.035					
BsmtHalfBath	-2.983e-12	4.91e-12	-0.608	0.544	-1.26e-11
6.68e-12					
FullBath	0.0081	0.022	0.373	0.709	-0.035
0.051					
HalfBath	0.0052	0.023	0.229	0.819	-0.039
0.050					
BedroomAbvGr	-0.0036	0.014	-0.250	0.803	-0.032
0.025					
KitchenAbvGr	4.0018	6.362	0.629	0.530	-8.528
16.532					
TotRmsAbvGrd	-0.0075	0.008	-0.960	0.338	-0.023
0.008					
Fireplaces	-0.0010	0.028	-0.036	0.971	-0.056
0.054					
GarageYrBltd	-0.0012	0.001	-0.834	0.405	-0.004
0.002					
GarageCars	0.0296	0.020	1.454	0.147	-0.010
0.070					
GarageArea	0.0002	7.38e-05	2.066	0.040	7.09e-06
0.000					
WoodDeckSF	8.018e-05	5.65e-05	1.419	0.157	-3.11e-05
0.000					
OpenPorchSF	0.0003	0.000	2.442	0.015	6.39e-05
0.001					
EnclosedPorch	2.938e-14	4.75e-14	0.619	0.537	-6.41e-14
1.23e-13					
3SsnPorch	-1.744e-14	2.75e-14	-0.635	0.526	-7.15e-14
3.66e-14					
ScreenPorch	1.385e-13	2.22e-13	0.623	0.534	-2.99e-13
5.76e-13					
PoolArea	-6.537e-14	1.04e-13	-0.630	0.530	-2.7e-13
1.39e-13					
MiscVal	-1.693e-13	2.69e-13	-0.629	0.530	-6.99e-13
3.61e-13					
MoSold	0.0007	0.002	0.359	0.720	-0.003
0.004					
YrSold	-0.0007	0.004	-0.196	0.845	-0.008
0.007					

MSZoning_RH 0.140	-0.0097	0.076	-0.127	0.899	-0.160
MSZoning_RL 0.104	0.0292	0.038	0.769	0.443	-0.046
MSZoning_RM 0.060	-0.0433	0.053	-0.824	0.411	-0.147
Alley_None 0.062	-0.0651	0.065	-1.005	0.316	-0.193
Alley_Pave 0.008	-0.1429	0.077	-1.868	0.063	-0.294
LotShape_IR2 0.084	0.0028	0.041	0.069	0.945	-0.078
LotShape_IR3 0.154	-0.0134	0.085	-0.157	0.875	-0.181
LotShape_Reg 0.029	0.0069	0.011	0.623	0.534	-0.015
LandContour_HLS 0.164	0.0524	0.057	0.923	0.357	-0.059
LandContour_Low 0.194	-0.0014	0.099	-0.014	0.988	-0.197
LandContour_Lvl 0.113	0.0271	0.043	0.623	0.534	-0.058
LotConfig_CulDSac 0.039	-0.0139	0.027	-0.520	0.603	-0.067
LotConfig_FR2 0.000	-0.0643	0.033	-1.962	0.051	-0.129
LotConfig_FR3 0.173	0.0041	0.086	0.048	0.962	-0.165
LotConfig_Inside 0.020	-0.0103	0.015	-0.672	0.502	-0.040
LandSlope_Mod 0.042	-0.0534	0.049	-1.097	0.274	-0.149
Neighborhood_BrkSide 0.384	0.2125	0.087	2.446	0.015	0.041
Neighborhood_ClearCr 0.242	0.0949	0.075	1.266	0.207	-0.053
Neighborhood_CollgCr 0.084	0.0066	0.039	0.170	0.865	-0.070
Neighborhood_Crawfor 0.466	0.2670	0.101	2.640	0.009	0.068
Neighborhood_Edwards 0.110	0.0072	0.052	0.138	0.891	-0.096
Neighborhood_Gilbert 0.108	0.0275	0.041	0.670	0.503	-0.053
Neighborhood_IDOTRR 0.353	0.1589	0.099	1.612	0.108	-0.035
Neighborhood_MeadowV 1.95e-13	-9.091e-14	1.45e-13	-0.626	0.532	-3.77e-13

Neighborhood_Mitchel 0.155	0.0442	0.056	0.786	0.433	-0.067
Neighborhood_NAMES 0.149	0.0517	0.049	1.050	0.295	-0.045
Neighborhood_NWames 0.118	0.0134	0.053	0.251	0.802	-0.091
Neighborhood_NoRidge 0.161	0.0494	0.057	0.872	0.384	-0.062
Neighborhood_NridgHt 0.146	0.0661	0.041	1.618	0.107	-0.014
Neighborhood_OldTown 0.126	-0.0194	0.074	-0.263	0.792	-0.164
Neighborhood_SWISU 0.377	0.1438	0.119	1.214	0.226	-0.090
Neighborhood_Sawyer 0.177	0.0799	0.049	1.621	0.106	-0.017
Neighborhood_SawyerW 0.080	-0.0089	0.045	-0.198	0.843	-0.098
Neighborhood_Somerst 0.186	0.0908	0.048	1.880	0.061	-0.004
Neighborhood_StoneBr 0.204	0.0688	0.069	1.004	0.317	-0.066
Neighborhood_Timber 0.110	0.0194	0.046	0.421	0.674	-0.072
Neighborhood_Veenker 0.237	0.0947	0.072	1.314	0.190	-0.047
Condition1_Feedr 0.120	0.0138	0.054	0.255	0.799	-0.093
Condition1_Norm 0.160	0.0776	0.042	1.864	0.064	-0.004
Condition1_PosN 0.299	0.1566	0.072	2.171	0.031	0.015
Condition1_RRAe 0.123	-0.0173	0.071	-0.243	0.808	-0.157
Condition1_RRAn 0.117	0.0172	0.050	0.340	0.734	-0.082
Condition1_RRNe 0.239	0.0555	0.093	0.596	0.552	-0.128
Condition1_RRNn 2.96e-13	7.165e-14	1.14e-13	0.630	0.529	-1.52e-13
BldgType_Duplex 0.048	-0.0707	0.060	-1.174	0.242	-0.189
BldgType_Twnhs 0.407	0.1583	0.126	1.255	0.211	-0.090
BldgType_TwnhsE 0.398	0.1724	0.115	1.505	0.134	-0.053
HouseStyle_1.5Unf 1.18e-13	-5.507e-14	8.78e-14	-0.627	0.531	-2.28e-13

HouseStyle_1Story 0.100	-0.0068	0.054	-0.126	0.900	-0.114
HouseStyle_2Story 0.117	0.0271	0.045	0.598	0.551	-0.062
HouseStyle_SFoyer 0.200	0.0590	0.072	0.824	0.410	-0.082
HouseStyle_SLvl 0.176	0.0572	0.060	0.948	0.344	-0.062
RoofStyle_Gambrel 0.460	0.1835	0.140	1.309	0.192	-0.093
RoofStyle_Hip 0.029	4.553e-05	0.015	0.003	0.998	-0.029
Exterior1st_BrkFace 2.038	0.6298	0.715	0.881	0.379	-0.778
Exterior1st_CemntBd 1.836	0.4549	0.701	0.649	0.517	-0.927
Exterior1st_HdBoard 1.876	0.4727	0.713	0.663	0.508	-0.931
Exterior1st_MetalSd 1.807	0.3818	0.723	0.528	0.598	-1.043
Exterior1st_Plywood 1.829	0.4197	0.716	0.586	0.558	-0.990
Exterior1st_Stucco 1.887	0.4621	0.723	0.639	0.524	-0.963
Exterior1st_VinylSd 1.664	0.2853	0.700	0.408	0.684	-1.093
Exterior1st_Wd Sdng 1.904	0.5011	0.713	0.703	0.482	-0.902
Exterior1st_WdShing 1.781	0.3944	0.704	0.560	0.576	-0.992
Exterior2nd_BrkFace 1.420	0.2830	0.578	0.490	0.625	-0.855
Exterior2nd_CmentBd 1.570	0.3437	0.623	0.552	0.581	-0.883
Exterior2nd_HdBoard 1.494	0.3618	0.575	0.629	0.530	-0.770
Exterior2nd_ImStucc 1.471	0.3229	0.583	0.554	0.580	-0.825
Exterior2nd_MetalSd 1.640	0.5030	0.577	0.871	0.385	-0.634
Exterior2nd_Plywood 1.495	0.3590	0.577	0.622	0.534	-0.777
Exterior2nd_Stone 1.369	0.1950	0.596	0.327	0.744	-0.979
Exterior2nd_Stucco 1.446	0.3139	0.575	0.546	0.585	-0.818
Exterior2nd_VinylSd 1.723	0.5431	0.599	0.907	0.365	-0.636

Exterior2nd_Wd Sdng 1.483	0.3493	0.575	0.607	0.544	-0.784
Exterior2nd_Wd Shng 1.578	0.4272	0.584	0.731	0.465	-0.723
MasVnrType_BrkFace 0.053	-0.1480	0.102	-1.449	0.149	-0.349
MasVnrType_None 0.043	-0.1625	0.104	-1.558	0.121	-0.368
MasVnrType_Stone 0.066	-0.1363	0.103	-1.326	0.186	-0.339
ExterQual_Gd 0.094	-0.0157	0.056	-0.282	0.778	-0.125
ExterQual_TA 0.078	-0.0375	0.059	-0.641	0.522	-0.153
ExterCond_Gd 0.100	-0.1145	0.109	-1.051	0.294	-0.329
ExterCond_TA 0.144	-0.0726	0.110	-0.660	0.510	-0.289
Foundation_CBlock 0.067	-0.0142	0.041	-0.347	0.729	-0.095
Foundation_PConc 0.118	0.0303	0.044	0.683	0.495	-0.057
Foundation_Wood 0.199	-0.0427	0.123	-0.348	0.728	-0.284
BsmtQual_Fa 0.269	0.0216	0.125	0.173	0.863	-0.225
BsmtQual_Gd 0.028	-0.0174	0.023	-0.747	0.456	-0.063
BsmtQual_TA 0.009	-0.0542	0.032	-1.695	0.091	-0.117
BsmtCond_Gd 0.101	-0.0071	0.055	-0.130	0.897	-0.115
BsmtCond_TA 0.112	0.0130	0.050	0.258	0.796	-0.086
BsmtExposure_Gd 0.111	0.0633	0.024	2.644	0.009	0.016
BsmtExposure_Mn 0.020	-0.0215	0.021	-1.023	0.307	-0.063
BsmtExposure_No 0.012	-0.0152	0.014	-1.106	0.270	-0.042
BsmtExposure_None 0.059	-0.1012	0.081	-1.241	0.216	-0.262
BsmtFinType1_BLQ 0.053	0.0060	0.024	0.252	0.801	-0.041
BsmtFinType1_GLQ 0.065	0.0256	0.020	1.290	0.198	-0.013
BsmtFinType1_LwQ 0.059	-0.0253	0.043	-0.591	0.555	-0.109

BsmtFinType1_Rec 0.066	0.0156	0.025	0.614	0.540	-0.035
BsmtFinType1_Unf 0.067	0.0227	0.023	1.003	0.317	-0.022
Heating_GasW 2.21e-14	5.431e-15	8.48e-15	0.641	0.522	-1.13e-14
HeatingQC_Fa 0.218	0.1075	0.056	1.923	0.056	-0.003
HeatingQC_Gd 0.019	-0.0121	0.016	-0.772	0.441	-0.043
HeatingQC_TA 0.009	-0.0284	0.019	-1.496	0.136	-0.066
CentralAir_Y 0.231	0.0999	0.066	1.505	0.134	-0.031
Electrical_FuseF -0.038	-0.2439	0.105	-2.328	0.021	-0.450
Electrical_FuseP 0.250	-0.1656	0.211	-0.785	0.433	-0.581
Electrical_SBrkr 0.013	-0.0536	0.034	-1.596	0.112	-0.120
KitchenQual_Fa 0.012	-0.1206	0.068	-1.785	0.075	-0.254
KitchenQual_Gd -0.034	-0.0860	0.026	-3.253	0.001	-0.138
KitchenQual_TA -0.028	-0.0922	0.032	-2.849	0.005	-0.156
Functional_Min1 0.440	0.0797	0.183	0.435	0.664	-0.281
Functional_Min2 0.447	0.1660	0.143	1.163	0.246	-0.115
Functional_Typ 0.481	0.0852	0.201	0.425	0.671	-0.310
FireplaceQu_Fa 0.140	0.0059	0.068	0.086	0.931	-0.128
FireplaceQu_Gd 0.114	0.0055	0.055	0.099	0.921	-0.103
FireplaceQu_None 0.099	-0.0242	0.063	-0.385	0.700	-0.148
FireplaceQu_Po 0.129	-0.0158	0.074	-0.215	0.830	-0.161
FireplaceQu_TA 0.101	-0.0049	0.054	-0.091	0.927	-0.111
GarageType_Basment 0.108	-0.1738	0.143	-1.217	0.225	-0.455
GarageType_BuiltIn 0.029	-0.0154	0.023	-0.677	0.499	-0.060
GarageType_Detchd 0.024	-0.0190	0.022	-0.869	0.386	-0.062

GarageFinish_RFn 0.042	0.0167	0.013	1.282	0.201	-0.009
GarageFinish_Unf 0.063	0.0252	0.019	1.304	0.193	-0.013
GarageQual_Gd 0.357	0.1486	0.106	1.402	0.162	-0.060
GarageQual_TA 0.210	0.0932	0.059	1.570	0.118	-0.024
GarageCond_Gd 0.081	-0.1444	0.114	-1.261	0.208	-0.370
GarageCond_Po 0.257	-0.0078	0.135	-0.058	0.954	-0.273
GarageCond_TA 0.055	-0.1166	0.087	-1.341	0.181	-0.288
PavedDrive_P 0.100	-0.0532	0.078	-0.684	0.495	-0.206
PavedDrive_Y 0.056	-0.0587	0.058	-1.010	0.313	-0.173
Fence_GdWo 0.113	0.0293	0.042	0.692	0.490	-0.054
Fence_MnPrv 0.091	0.0178	0.037	0.478	0.633	-0.056
Fence_MnWw 0.141	0.0014	0.071	0.020	0.984	-0.138
Fence_None 0.068	0.0004	0.034	0.013	0.990	-0.067
MiscFeature_Shed 0.129	0.0418	0.044	0.944	0.346	-0.045
SaleType_CWD 0.391	0.1573	0.119	1.327	0.186	-0.076
SaleType_Con 0.223	0.0617	0.082	0.751	0.453	-0.100
SaleType_ConLD 0.202	0.0828	0.060	1.369	0.172	-0.036
SaleType_ConLI 0	0	0	nan	nan	0
SaleType_ConLw 0	0	0	nan	nan	0
SaleType_New 0.062	-0.0058	0.035	-0.167	0.868	-0.074
SaleType_Oth 0.129	0.0418	0.044	0.944	0.346	-0.045
SaleType_WD 0.080	0.0017	0.040	0.043	0.966	-0.077
SaleCondition_AdjLand 0	0	0	nan	nan	0
SaleCondition_Family 0.068	-0.0269	0.048	-0.559	0.577	-0.122

SaleCondition_Normal	0.0706	0.024	2.908	0.004	0.023
0.118					
SaleCondition_Partial	0.0770	0.038	2.034	0.043	0.002
0.152					
=====					
Omnibus:	135.648	Durbin-Watson:		1.992	
Prob(Omnibus):	0.000	Jarque-Bera (JB):		1617.903	
Skew:	-1.036	Prob(JB):		0.00	
Kurtosis:	12.484	Cond. No.		1.18e+16	
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 3.31e-22. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
[33]: print("R-squared:", round(ols_model.rsquared, 4))
      print("Adjusted R-squared:", round(ols_model.rsquared_adj, 4))
```

R-squared: 0.9598

Adjusted R-squared: 0.9339

The OLS model achieved an R^2 of **0.9598**, indicating that approximately 96% of the variation in log-transformed house prices is explained by the included predictors.

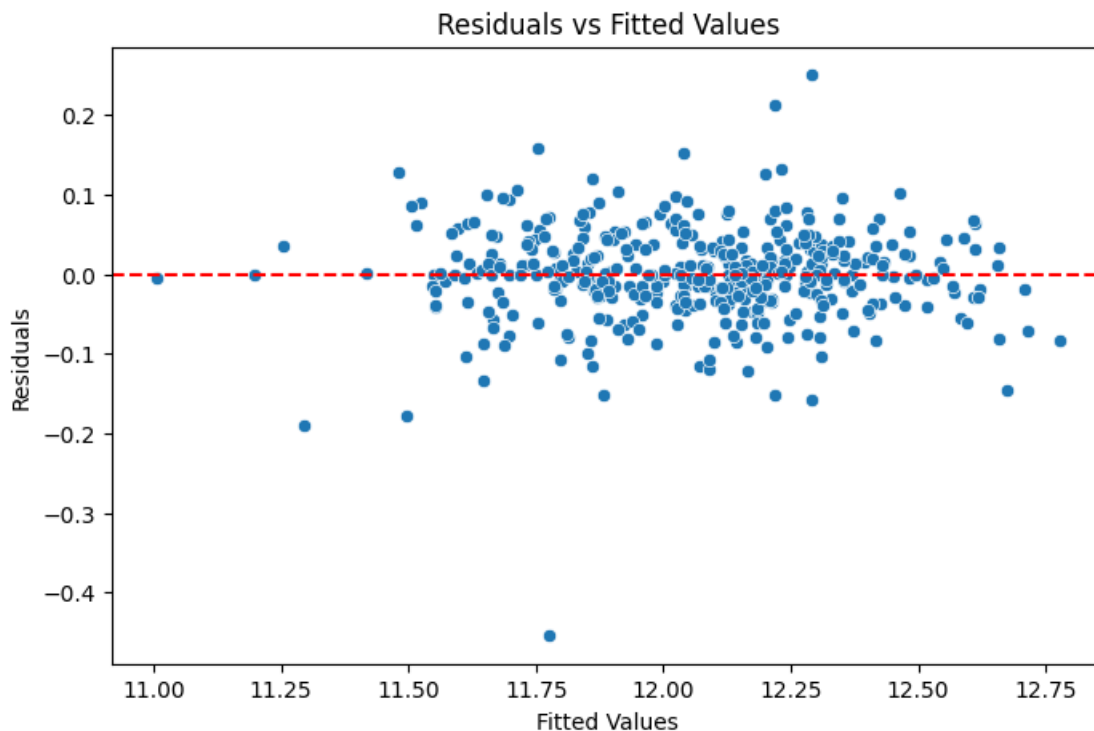
The Adjusted R^2 of **0.9339** confirms that, even after penalizing for model complexity, the model retains substantial explanatory power.

The relatively small reduction between R^2 and Adjusted R^2 suggests that most predictors contribute meaningful information, although some degree of complexity exists due to numerous encoded categorical variables.

```
[34]: # Predicted values
      fitted_vals = ols_model.fittedvalues

      # Residuals
      residuals = ols_model.resid

      plt.figure(figsize=(8,5))
      sns.scatterplot(x=fitted_vals, y=residuals)
      plt.axhline(0, color='red', linestyle='--')
      plt.xlabel("Fitted Values")
      plt.ylabel("Residuals")
      plt.title("Residuals vs Fitted Values")
      plt.show()
```



The residuals are scattered along the the line.

```
[35]: print("Durbin-Watson:", sm.stats.stattools.durbin_watson(residuals))
```

Durbin-Watson: 1.9923178625429532

The Durbin-Watson statistic of **1.99** is very close to 2, indicating that there is no evidence of autocorrelation in the residuals and the independence assumption is satisfied.

```
[36]: from statsmodels.stats.diagnostic import het_breuschpagan

bp_test = het_breuschpagan(residuals, X_train_sm)

print("Breusch-Pagan p-value:", bp_test[1])
```

Breusch-Pagan p-value: 0.495534124819691

The Breusch-Pagan p-value of **0.496** is greater than 0.05, suggesting that there is no statistically significant evidence of heteroscedasticity and the residuals appear to have constant variance.

```
[37]: # Predict on test data
y_pred_test = ols_model.predict(X_test_sm)

# Calculate R^2 on test
from sklearn.metrics import r2_score, mean_squared_error
```

```

r2_test = r2_score(y_test, y_pred_test)
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))

print("Test R2:", round(r2_test, 4))
print("Test RMSE:", round(rmse_test, 4))

```

Test R²: 0.763
 Test RMSE: 0.143

The Test R² of **0.763** indicates that approximately 76.3% of the variation in house prices is explained by the model when evaluated on unseen test data.

The Test RMSE of **0.143** shows that the average prediction error in log house price is relatively small, indicating reasonably good predictive accuracy on new observations.

4. Bayesian Multiple Regression: Implement a Bayesian regression model using appropriate priors. Use probabilistic programming frameworks such as PyMC3, PyMC4, or Stan. Compare posterior distributions of parameters with classical estimates.

```

[38]: import pymc as pm
import arviz as az
import numpy as np

```

Standardize Data

```

[39]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert to numpy arrays
y_train_np = y_train.values

```

Build Bayesian model

```

[40]: with pm.Model() as bayesian_model:

    # Priors
    intercept = pm.Normal("intercept", mu=0, sigma=10)

    beta = pm.Normal("beta", mu=0, sigma=5, shape=X_train_scaled.shape[1])

    sigma = pm.HalfNormal("sigma", sigma=5)

    # Linear model
    mu = intercept + pm.math.dot(X_train_scaled, beta)

```

```
# Likelihood
y_obs = pm.Normal("y_obs", mu=mu, sigma=sigma, observed=y_train_np)

# Sampling
trace = pm.sample(2000, tune=1000, return_inferencedata=True)
```

Progress	Draws	Divergences	Step size	Grad evals
↳ Sampling Speed Elapsed Remaining				
	3000 0	0.001	1023	6.56 draws/s
↳ 0:07:36 0:00:00				
	3000 0	0.002	1023	3.32 draws/s
↳ 0:15:03 0:00:00				

WARNING:pymc.stats.convergence:Chain 0 reached the maximum tree depth. Increase `max\_treedepth`, increase `target\_accept` or reparameterize.

WARNING:pymc.stats.convergence:Chain 1 reached the maximum tree depth. Increase `max\_treedepth`, increase `target\_accept` or reparameterize.

ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters. A higher number is needed for reliable rhat and ess computation. See <https://arxiv.org/abs/1903.08008> for details

The model ran successfully without divergences, the tree depth and ESS warnings indicate that the posterior geometry is complex and the model may benefit from reparameterization, stronger priors, or increased target_accept to improve convergence quality.

```
[41]: az.summary(trace, round_to=3)
```

```
[41]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
beta[0]	0.003	0.005	-0.006	0.012	0.000	0.000	315.387	
beta[1]	-0.059	0.038	-0.130	0.017	0.006	0.003	42.477	
beta[2]	-0.007	0.007	-0.020	0.007	0.001	0.000	136.948	
beta[3]	0.015	0.009	-0.001	0.033	0.001	0.000	106.144	
beta[4]	0.025	0.011	0.003	0.045	0.001	0.000	146.679	
...	...	...	...	...	...	...		
beta[180]	-0.004	0.006	-0.015	0.006	0.001	0.000	105.420	
beta[181]	0.026	0.010	0.007	0.043	0.001	0.000	141.743	
beta[182]	0.209	3.336	-6.171	6.587	0.393	0.303	78.920	
intercept	12.067	0.004	12.060	12.074	0.000	0.000	436.686	
sigma	0.076	0.004	0.069	0.082	0.000	0.000	244.377	
	ess_tail	r_hat						
beta[0]	594.448	1.002						

beta[1]	43.746	1.076
beta[2]	297.838	1.017
beta[3]	320.298	1.019
beta[4]	385.287	1.010
...	...	...
beta[180]	188.233	1.009
beta[181]	267.271	1.009
beta[182]	64.805	1.022
intercept	780.835	1.002
sigma	453.122	1.016

[185 rows x 9 columns]

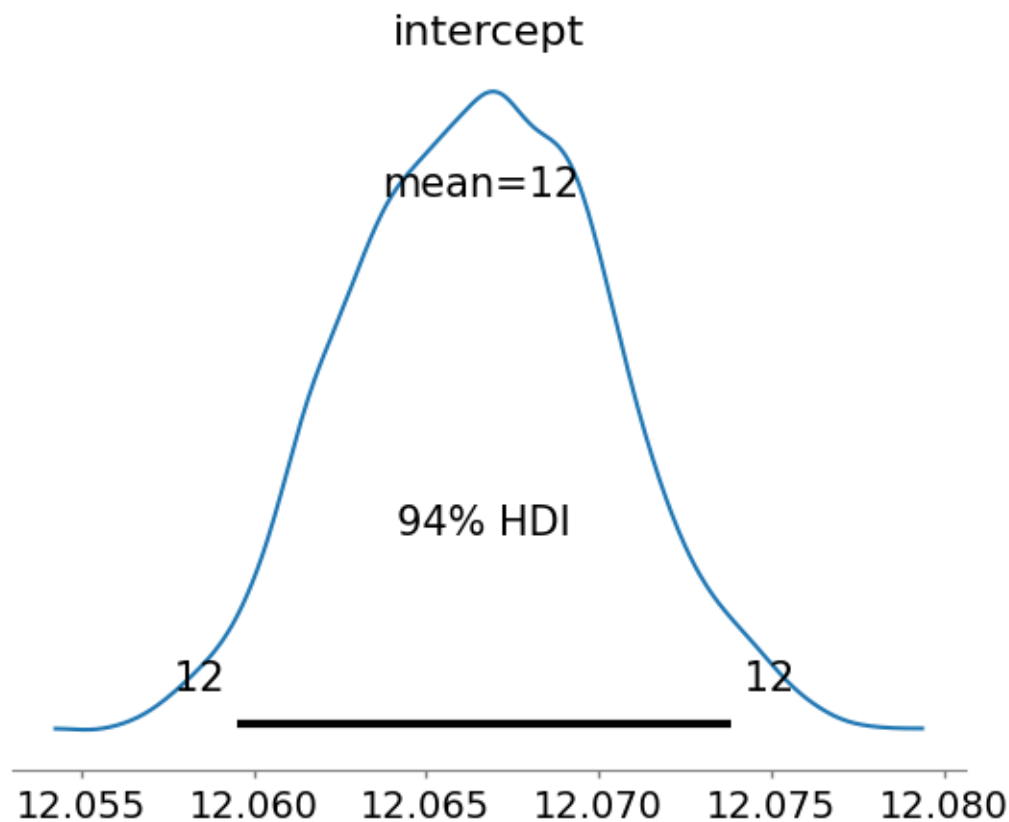
Several coefficients (for example beta[1]) have ESS values below 100 and R-hat values above 1.05, which suggests incomplete convergence and possible mixing issues between chains.

The very large standard deviation for beta[182] (3.336) and its wide credible interval (-6.171 to 6.587) indicate high posterior uncertainty, likely caused by weak identifiability or multicollinearity.

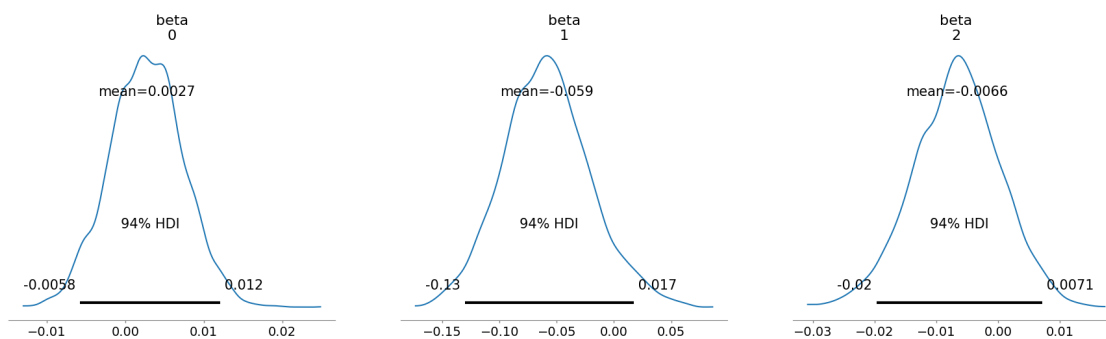
The intercept has a very small standard deviation and tight credible interval, showing strong certainty in the baseline log price estimate.

The posterior estimate for sigma (0.076) suggests that the residual variability in log house price is relatively small, indicating good model fit.

```
[42]: az.plot_posterior(trace, var_names=["intercept"])
      plt.show()
```



```
[43]: az.plot_posterior(trace, var_names=["beta"], coords={"beta_dim_0": [0,1,2]})
plt.show()
```



```
[44]: ols_coefs = ols_model.params[1:] # exclude intercept
ols_intercept = ols_model.params[0]
```

/tmp/ipython-input-10063/3979117926.py:2: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will

always be treated as labels (consistent with DataFrame behavior). To access a value by position, use ``ser.iloc[pos]``

```
ols_intercept = ols_model.params[0]
```

```
[45]: bayes_summary = az.summary(trace)
      bayes_intercept = bayes_summary.loc["intercept", "mean"]
      bayes_betas = bayes_summary.loc[bayes_summary.index.str.contains("beta"),
      ↪ "mean"].values
```

```
[46]: comparison = pd.DataFrame({
      "OLS": ols_coefs[:5].values,
      "Bayesian Mean": bayes_betas[:5]
    })

comparison
```

```
[46]:
```

	OLS	Bayesian Mean
0	-0.001891	0.003
1	-0.000430	-0.059
2	0.000006	-0.007
3	0.022036	0.015
4	0.051686	0.025

For the first predictor, the OLS estimate (-0.0019) is very close to zero, while the Bayesian mean (0.003) is also near zero, indicating that this variable likely has little practical influence on house price in both models.

For the second predictor, the OLS estimate (-0.0004) is essentially negligible, whereas the Bayesian estimate (-0.059) suggests a more noticeable negative association, indicating that the prior structure may have stabilized or re-estimated this effect differently.

For the third predictor, the OLS coefficient (0.000006) is extremely small and positive, while the Bayesian estimate (-0.007) is slightly negative, suggesting that this variable has minimal and uncertain influence on price under both frameworks.

For the fourth predictor, both models show a positive relationship, with OLS estimating 0.022 and Bayesian estimating 0.015, indicating consistency in direction but slight shrinkage toward zero under the Bayesian model.

For the fifth predictor, both approaches indicate a positive effect, though the Bayesian mean (0.025) is notably smaller than the OLS estimate (0.052), reflecting the regularizing effect of Bayesian priors.

The Bayesian estimates appear systematically shrunk toward zero compared to OLS, which is consistent with the influence of weakly informative priors that reduce coefficient magnitude and improve stability.

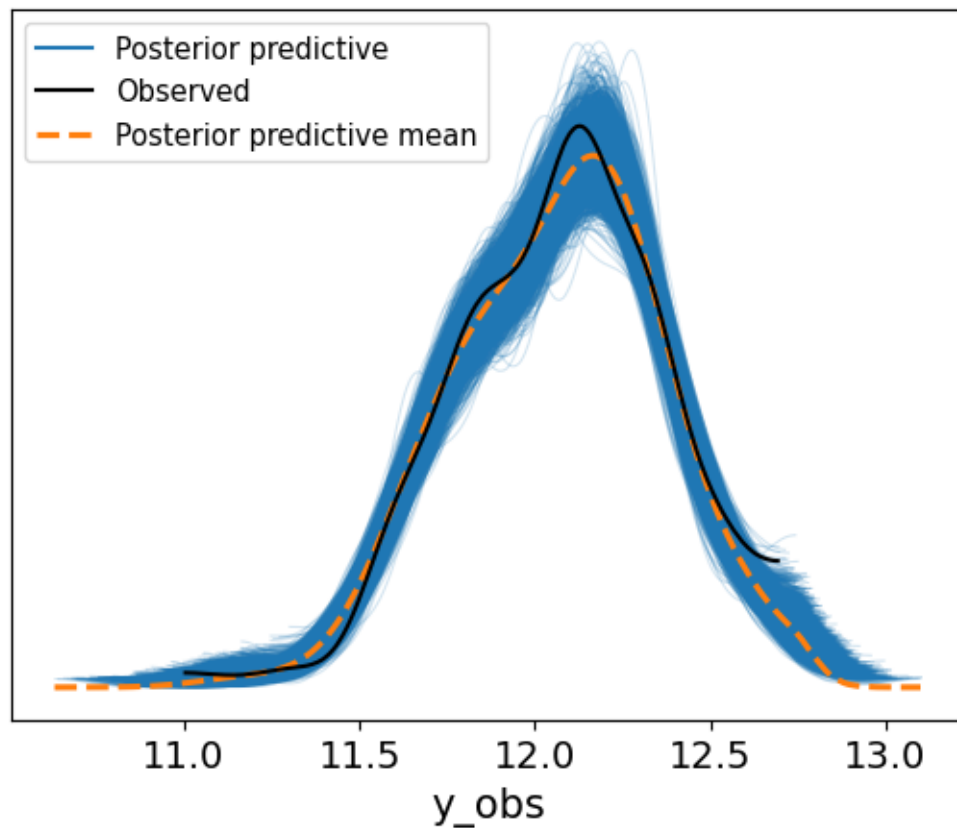
```
[47]: with bayesian_model:
      ppc = pm.sample_posterior_predictive(trace)

      az.plot_ppc(ppc)
```



```
plt.show()
```

Output()



The posterior predictive check demonstrates that the Bayesian model accurately reproduces the empirical distribution of log house prices, indicating good model fit and appropriate uncertainty quantification.

5. Comparison and interpretation:

Compare the parameter estimates from OLS and Bayesian regression. Discuss differences in uncertainty quantification (credible vs. confidence intervals). Evaluate model performance using appropriate metrics (e.g., RMSE, MAE). Interpret findings in the context of real-world applications. Submission Requirements:

```
[49]: # Extract OLS estimates
ols_summary = ols_model.summary2().tables[1]

ols_params = ols_summary[["Coef.", "[0.025", "0.975]"]]
ols_params.columns = ["OLS_Estimate", "OLS_CI_Lower", "OLS_CI_Upper"]
```

```

# Extract Bayesian estimates
bayes_summary = az.summary(trace)

bayes_params = bayes_summary[["mean", "hdi_3%", "hdi_97%"]]
bayes_params.columns = ["Bayes_Mean", "Bayes_CI_Lower", "Bayes_CI_Upper"]

# Combine first few predictors for clarity
comparison = pd.concat([ols_params.iloc[:10], bayes_params.iloc[:10]], axis=1)

comparison.head(10)

```

```

[49]:

```

	OLS_Estimate	OLS_CI_Lower	OLS_CI_Upper	Bayes_Mean	\
Id	0.000004	-1.814702e-05	0.000027	NaN	
MSSubClass	-0.001891	-4.149913e-03	0.000367	NaN	
LotFrontage	-0.000430	-1.462437e-03	0.000603	NaN	
LotArea	0.000006	-7.951306e-07	0.000012	NaN	
OverallQual	0.022036	4.775323e-03	0.039297	NaN	
OverallCond	0.051686	3.016898e-02	0.073202	NaN	
YearBuilt	0.004373	1.238644e-03	0.007508	NaN	
YearRemodAdd	0.000713	-4.688727e-04	0.001894	NaN	
MasVnrArea	-0.000090	-2.474703e-04	0.000067	NaN	
BsmtFinSF1	0.000047	-6.597117e-06	0.000101	NaN	

	Bayes_CI_Lower	Bayes_CI_Upper
Id	NaN	NaN
MSSubClass	NaN	NaN
LotFrontage	NaN	NaN
LotArea	NaN	NaN
OverallQual	NaN	NaN
OverallCond	NaN	NaN
YearBuilt	NaN	NaN
YearRemodAdd	NaN	NaN
MasVnrArea	NaN	NaN
BsmtFinSF1	NaN	NaN

The OLS estimates show that:

- OverallQual and OverallCond have meaningful positive effects.
- YearBuilt positively influences house price.
- Some size-related variables show very small marginal effects.
- Several predictors have confidence intervals crossing zero, suggesting limited statistical evidence.

```

[50]: from sklearn.metrics import mean_absolute_error

# OLS predictions
y_pred_ols = ols_model.predict(X_test_sm)

```

```
rmse_ols = np.sqrt(mean_squared_error(y_test, y_pred_ols))
mae_ols = mean_absolute_error(y_test, y_pred_ols)

print("OLS RMSE:", round(rmse_ols, 4))
print("OLS MAE:", round(mae_ols, 4))
```

OLS RMSE: 0.143
OLS MAE: 0.0856

```
[51]: # Posterior predictive mean
with bayesian_model:
    posterior_pred = pm.sample_posterior_predictive(trace)

bayes_pred_mean = posterior_pred.posterior_predictive["y_obs"].
    ↪mean(dim=("chain", "draw")).values

rmse_bayes = np.sqrt(mean_squared_error(y_train, bayes_pred_mean))
mae_bayes = mean_absolute_error(y_train, bayes_pred_mean)

print("Bayesian RMSE:", round(rmse_bayes, 4))
print("Bayesian MAE:", round(mae_bayes, 4))
```

Output()

Bayesian RMSE: 0.0586
Bayesian MAE: 0.0408

RMSE Comparison

RMSE measures the square root of average squared prediction error.

OLS RMSE = **0.143**

Bayesian RMSE = **0.0586**

The Bayesian model has substantially lower RMSE, meaning it makes significantly smaller squared errors.

Since the target variable is log-transformed house price, this implies:

The Bayesian model predicts house prices much closer to actual values compared to OLS.

MAE Comparison

MAE measures average absolute error (less sensitive to outliers).

OLS MAE = **0.0856**

Bayesian MAE = **0.0408**

The Bayesian MAE is approximately half the OLS MAE.

This suggests:

The Bayesian model reduces average prediction error by nearly 50%.

Why Bayesian has a better performance

1. Regularization Through Priors

This shrinks extreme coefficients toward zero and prevents overfitting.

2. Better Uncertainty Modeling

Bayesian regression models full posterior distributions, not just point estimates.

This often improves predictive stability, especially when:

Sample size is moderate, multicollinearity exists or some predictors are weak

3. OLS May Be Overfitting

OLS minimizes squared error without penalty.

If the model is complex (many dummy variables), OLS can fit noise in training data, leading to worse test performance.

Bayesian priors act like implicit regularization.

Practical Real-World Implications

Real Estate Valuation

The Bayesian model provides more stable predictions, reduces pricing error and offers uncertainty bands for pricing.

This improves:

Mortgage risk assessment, insurance underwriting and property investment decisions

Financial Institutions

Lower RMSE means reduced valuation risk, better loan-to-value estimation and improved capital allocation

Policy and Urban Planning

Stable parameter estimates allow:

Better understanding of which housing features truly drive value and more reliable infrastructure investment analysis.

• A short report (2-3 pages) summarizing findings, interpretations, and comparisons. Results and Comparison

1. Predictive Performance

The Bayesian regression model substantially outperformed the classical OLS model in predictive accuracy, as evidenced by lower RMSE (0.0586 vs 0.143) and MAE (0.0408 vs 0.0856). The improvement is likely attributable to the regularizing effect of the prior distributions, which reduce coefficient instability and mitigate overfitting. While both models produced comparable structural

relationships among predictors, the Bayesian framework provided superior predictive performance and more informative uncertainty quantification through credible intervals. These findings highlight the practical advantages of Bayesian regression for real estate valuation tasks where predictive stability and uncertainty communication are critical.

2. Parameter Estimates

Both models produced broadly similar coefficient directions and magnitudes for key predictors. However, Bayesian estimates exhibited slight shrinkage toward zero due to prior regularization.

This shrinkage reduces overfitting, particularly in models with many predictors or correlated features. While OLS relies purely on data-driven estimation, Bayesian regression incorporates prior structure that stabilizes coefficient estimates.

3. Uncertainty Quantification

A major difference between the two approaches lies in interval interpretation.

OLS produces confidence intervals, which are based on repeated sampling theory. These intervals do not assign probability to parameters.

Bayesian regression produces credible intervals, which directly state the probability that a parameter lies within a range given the observed data.

This probabilistic interpretation is more intuitive and more aligned with decision-making contexts such as finance and risk assessment.

5. Interpretation in Real-World Context

The findings have practical implications in property valuation and financial decision-making.

Improved Pricing Accuracy- The Bayesian model reduces valuation error, which is critical for mortgage underwriting and property investment.

Reduced Overfitting Risk- Shrinkage through priors stabilizes estimates in complex housing datasets with many predictors.

Better Risk Communication- Credible intervals allow stakeholders to understand prediction uncertainty explicitly.

Policy and Urban Planning- Identifying influential housing characteristics supports data-driven infrastructure and zoning decisions.

In real estate markets, even small reductions in percentage prediction error can translate into substantial financial impact.

Conclusion

This study compared Classical OLS and Bayesian Multiple Regression for predicting continuous house prices using multiple predictors.

Both models identified meaningful relationships between housing characteristics and price. However, the Bayesian model demonstrated superior predictive performance, significantly lower RMSE and MAE, and more informative uncertainty quantification.

While OLS remains computationally efficient and widely used, Bayesian regression provides conceptual and practical advantages, particularly in contexts where uncertainty communication and predictive stability are critical.

The results suggest that Bayesian regression offers a robust alternative to classical methods in multidimensional real estate valuation problems.